Module 3, Lecture 2 — Production-Level Entity-Relationship Design

Course: Staff Engineer / Technical Lead Mentorship · Module 3 · Lecture 2 of 3 · Case study: Spendwise

Part 1 — Learning objective

After this lecture you should be able to:

1. Read a set of requirements and systematically extract the entities, relationships, and rules that must be represented in the database.
2. Distinguish the three levels of ERD (conceptual, logical, physical) and produce the physical level that gets built.
3. Choose PostgreSQL column types correctly: money as integer minor units, dates vs timestamps, text without arbitrary length caps, UUIDs, SMALLINTs for small reference lists.
4. Apply the rule "NOT NULL is the default; add NULL only with a stated reason" and explain what each nullable column means.
5. Write CHECK constraints, foreign keys with the correct ON DELETE behaviour, and UNIQUE constraints that enforce business rules at the database level.
6. Design indexes matched to the real queries in the system, using partial indexes for filtered columns, and avoid over-indexing.
7. Apply four structural patterns correctly: soft delete, audit columns, fixed reference data as a seed table, and token families.
8. Write migration-safe DDL: what you can do safely on a live table, what requires care (CREATE INDEX CONCURRENTLY, expand-and-contract), and what you must never do on a column that other code depends on.
9. Trace every column in your schema back to a requirement or a design pattern, and remove anything that cannot be traced.

Part 2 — Real-world scenario

The TDD from Lecture 1 has been reviewed and approved by Meera. The ADRs are on record. You open the repository.

The TDD listed the entities in one line:

> Entities: users, categories (fixed list, stored as data), expenses, refresh_tokens (hashed), usage_events, deletion_requests.

That line is an intention without commitment. The ERD is the commitment. Until it exists, every developer has a different picture in their head of what `refresh_tokens` contains, whether categories are a table or an enum, what `deletion_requests` tracks, and how the joins work. When the pictures differ, the code diverges. That divergence shows up as a production bug at the worst possible moment.

The ERD's job is to make the picture shared and unambiguous. Your job in this lecture is to build it from first principles, not from intuition or from what the last project used.

Scope note: Lecture 1 established the architecture, component boundaries, key data flows, and the TDD and ADRs. This lecture produces the physical entity-relationship design: the tables, columns, constraints, keys, indexes, and schema evolution strategy. The full API contract is the subject of Lecture 3.

Part 3 — The lead's thought process

3.1 Three levels of ERD and why only one ships

Conceptual ERD. Boxes for entities (User, Expense, Category), lines for relationships, no types, no keys. A communication tool for a business conversation. You might sketch it in the first client meeting to confirm you have understood the domain.

Logical ERD. Adds attributes, identifies primary and foreign keys, shows cardinality and optionality, but uses generic types (String, Integer, Date) rather than database-specific ones. Useful for a team designing across multiple databases.

Physical ERD. The actual table names, column names, PostgreSQL types, nullability, DEFAULT values, CHECK constraints, and the indexes that will be created. This is the document that generates the migration scripts and the one code reviews check against. It is the only level that matters when building.

A lead produces the physical ERD, not a conceptual sketch with "we'll figure out the columns later." That phrase is how you get a TEXT column called `data` that stores JSON and is queried with a function that bypasses every index.

What an experienced lead checks in a physical ERD:

- every column has a type chosen for a reason;
- every nullable column has a stated reason for being nullable;
- every business rule that can be expressed as a constraint, is;
- every foreign key has an explicit ON DELETE behaviour;
- every index matches a real query.

3.2 Reading entities from requirements

The method is mechanical: read each requirement and ask, "what noun must the database remember to satisfy this?" One noun from one requirement usually maps to one entity (table).

Requirement	**Noun(s) the database must store**	**Maps to**
FR-1 Accounts with email and password	A user account, a credential	`users`
FR-2 Record / edit / delete expenses	An expense record	`expenses`
FR-3 Deleted expenses excluded from totals	The deletion state of an expense	`deleted_at` column on `expenses`
FR-4 Fixed categories	A fixed list of category names	`categories`
FR-5 Monthly summary by category	No new entity; computed from expenses	—
FR-9 Account deletion	A deletion request and its lifecycle	`deletion_requests`
FR-10 Usage events without financial content	An event record	`usage_events`
Flow 3 Sign-in / stay signed in	A refresh token (hashed)	`refresh_tokens`
NFR-5 Exactly once on retry	The expense ID (client-generated)	Primary key on `expenses`

NFR-7 Users see only own data	Ownership columns	`user_id` on every user-owned table
NFR-8 Backups expire in 35 days	No table; a backup retention setting	Configuration, not DDL

Result: six tables — `users`, `categories`, `expenses`, `refresh_tokens`, `usage_events`, `deletion_requests`. FR-3 (soft delete) is a column on `expenses`, not a separate table, because the deletion state belongs to the expense itself. FR-5 (monthly summary) produces no new table, because it is computed on read from `expenses`. If you cannot trace a table to a requirement, question whether it should exist.

3.3 Relationships, cardinality, and ON DELETE behaviour

Cardinality for Spendwise:

Relationship	Cardinality	Notes
User → Expenses	1:N	One user has many expenses
User → RefreshTokens	1:N	One user can have several active tokens (multiple devices)
User → DeletionRequest	1:1	A user has at most one pending deletion
User → UsageEvents	1:N	One user generates many events
Category → Expenses	1:N	One category appears on many expenses

No M:N relationships exist in this schema, which is why there are no junction tables.

ON DELETE behaviour is chosen per foreign key based on what the business requires when the parent row is deleted.

FK	ON DELETE choice	Reason
`expenses.user_id → users.id`	`CASCADE`	Account deletion must remove all expenses (NFR-8, FR-9). One statement per user, reliable.

`expenses.category_id → categories.id`	`RESTRICT`	Cannot delete a category that has expenses. Categories are a fixed list; RESTRICT guards against accidents.
`refresh_tokens.user_id → users.id`	`CASCADE`	A deleted account must not be able to sign in via a stale token.
`usage_events.user_id → users.id`	`SET NULL`	Retaining events for aggregate analytics even after account deletion is often useful. If privacy obligations require full erasure, use CASCADE instead — open question to confirm with Meera.
`deletion_requests.user_id → users.id`	`CASCADE`	If the user row is hard-deleted by the purge job, the request goes with it.

The default if you do not think about this is `NO ACTION`, which means "reject the delete if a child row exists." That is safe but wrong for expenses on account deletion — you would have to delete children before parents in application code, where it can be forgotten.

3.4 Column-level design decisions

Type selection. Use the tightest type that correctly represents the data.

Data	Recommended type	Common mistake	Why the mistake is wrong
Money amount	`BIGINT` (minor units, e.g. paise)	`FLOAT`	Floating point cannot represent `0.10` INR exactly; rounding errors accumulate across sums
Money amount	`BIGINT`	`NUMERIC(12,2)`	`NUMERIC` is correct but larger and slower; `BIGINT` with a divisor is both exact and compact. (`NUMERIC` is a defensible choice; `FLOAT` is not.)

Currency code	`CHAR(3)`	`TEXT`	ISO 4217 codes are always exactly 3 characters; `CHAR(3)` is self-documenting
Calendar date	`DATE`	`TIMESTAMP` or `TEXT`	`DATE` is exact; `TEXT` allows invalid values; `TIMESTAMP` carries time-of-day the product does not need (ADR-002)
Server time	`TIMESTAMPTZ`	`TIMESTAMP` (no TZ)	`TIMESTAMP` stores a bare clock reading; if the server's time zone setting changes, all values are misinterpreted. Always use `TIMESTAMPTZ`.
Primary key (UUID)	`UUID`	`TEXT` storing a UUID string	Native `UUID` is 16 bytes; a `TEXT` UUID is ~36 bytes. The difference compounds across millions of rows and index entries.
Small fixed list (`category_id`)	`SMALLINT` (2 bytes)	`INTEGER` or `UUID`	With at most a few dozen categories, `SMALLINT` is correct and compact. The FK column on expenses saves 6 bytes per row versus UUID.
Free text (`note`)	`TEXT` with a CHECK on `char_length`	`VARCHAR(200)`	PostgreSQL stores `TEXT` and `VARCHAR` identically. A CHECK constraint is more explicit than an implicit truncation rule.
Token hash	`TEXT`	`BYTEA`	A hex- or Base64-encoded hash is a text string; `TEXT` is appropriate.

Nullability. `NOT NULL` is the default. Add `NULL` only with a specific, stated reason.

Column	Nullable?	Reason
`expenses.note`	Yes	Notes are optional; the product allows an expense without a note
`expenses.deleted_at`	Yes	`NULL` means not deleted; a non-null timestamp means soft-deleted. The NULL carries semantic meaning.
`users.password_hash`	Yes (transitional)	ADR-005 is unresolved. Nullable during the decision period; set `NOT NULL` once the choice is final.
`refresh_tokens.revoked_at`	Yes	`NULL` means the token is live; non-null means revoked. Same pattern as `deleted_at`.
`usage_events.user_id`	Yes	To allow `SET NULL` on account deletion while retaining events for analytics.
Everything else	`NOT NULL`	No reason to allow NULL.

Audit columns. Every mutable table gets `created_at TIMESTAMPTZ NOT NULL DEFAULT now()` and `updated_at TIMESTAMPTZ NOT NULL DEFAULT now()`. Immutable and append-only tables (`categories`, `usage_events`, `deletion_requests`) need only `created_at`. Update `updated_at` with a trigger — not application discipline, which will be forgotten:

```
CREATE OR REPLACE FUNCTION set_updated_at()
RETURNS TRIGGER LANGUAGE plpgsql AS $$
BEGIN
  NEW.updated_at = now();
  RETURN NEW;
END;
$$;
```

Then on each mutable table:

```
CREATE TRIGGER trg_<table>_updated_at
BEFORE UPDATE ON <table>
FOR EACH ROW EXECUTE FUNCTION set_updated_at();
```

CHECK constraints. A CHECK constraint is the database's promise that a condition is always true for every row. It is the right place for business rules that apply to a single row and can be evaluated from the row's own columns.

Table	Constraint	Rule it enforces
`expenses`	`amount_minor > 0 AND amount_minor <= 1000000000`	Amounts are positive and within a sane upper bound (max 10,000,000.00 INR)
`expenses`	`char_length(note) <= 200`	Notes cannot exceed 200 characters
`expenses`	`currency_code ~ '^[A-Z]{3}$'`	Currency codes are three uppercase letters
`refresh_tokens`	`expires_at > created_at`	A token cannot expire before it was created
`usage_events`	`event_type IN ('app_opened', 'expense_saved', ...)`	Only known event types are accepted
`budget_alerts` (exercise)	`threshold IN (80, 100)`	Only valid thresholds
`budget_alerts` (exercise)	`EXTRACT(DAY FROM month_start) = 1`	`month_start` is always the first of a month

What does **NOT** belong in a CHECK constraint: anything referencing another table (use a foreign key), anything involving the current timestamp as a comparison (use triggers), and rules spanning multiple rows (use UNIQUE constraints or application code).

3.5 Reference data: the categories decision

"Fixed categories" is an interesting design decision worth its own paragraph. The options:

Option 1: PostgreSQL ENUM type.

```
CREATE TYPE expense_category AS ENUM ('food', 'transport', ...);
```

Type-safe and compact. The problem: DDL changes are required to add a value, removing or renaming a value is painful or impossible, and the list of valid values is hidden from SQL queries. **Rejected.**

Option 2: CHECK constraint on expenses.

```
CHECK (category_id IN (1, 2, 3, ...))
```

No separate table. The problem: the human-readable names live only in application code. A SQL query shows category IDs with no context. Adding a category requires a code deploy. **Rejected.**

Option 3: Reference table with seeded data.

A `categories(id SMALLINT, name TEXT, sort_order SMALLINT)` table populated once at migration time, referenced by a foreign key. Adding a new category is an `INSERT` (no DDL change). The name is visible in SQL joins. The FK enforces validity. This is the correct choice, and it is what the TDD meant by "stored as data."

The `SMALLINT` primary key matters: since `category_id` appears on every expense row and is part of composite indexes, its size directly affects storage and index performance. A `SMALLINT` is 2 bytes vs 4 bytes for `INTEGER` and 16 bytes for `UUID`. For a table with millions of rows this is not micro-optimisation.

Seeding. The initial category rows are inserted in a migration script, not in application code. The database is always in a valid state immediately after migration, regardless of whether the application has started.

3.6 The physical schema

Here is the complete DDL for all six tables. Read each column and find the decision from sections 3.2–3.5 that produced it.

```
-- ─── Shared trigger function
─────────────────────────────────────────────

CREATE OR REPLACE FUNCTION set_updated_at()
RETURNS TRIGGER LANGUAGE plpgsql AS $$
BEGIN
  NEW.updated_at = now();
  RETURN NEW;
END;
$$;

-- ─── 1. categories
─────────────────────────────────────────────────────

CREATE TABLE categories (
  id         SMALLINT     PRIMARY KEY
             GENERATED BY DEFAULT AS IDENTITY,
  name       TEXT         NOT NULL,
  sort_order SMALLINT     NOT NULL CHECK (sort_order >= 0),
  created_at TIMESTAMPTZ  NOT NULL DEFAULT now(),
  UNIQUE (name)
);

INSERT INTO categories (name, sort_order) VALUES
  ('Food & Drink', 1), ('Transport', 2), ('Utilities', 3),
  ('Health', 4), ('Shopping', 5), ('Entertainment', 6),
  ('Education', 7), ('Other', 8);

-- ─── 2. users
─────────────────────────────────────────────────────────

CREATE TABLE users (
  id                  UUID         PRIMARY KEY DEFAULT
gen_random_uuid(),
  email               TEXT         NOT NULL,
  password_hash       TEXT,                    -- nullable until ADR-
005 resolved
  pending_deletion_at TIMESTAMPTZ,             -- set immediately on
deletion request
  created_at          TIMESTAMPTZ  NOT NULL DEFAULT now(),
```

```
  updated_at          TIMESTAMPTZ  NOT NULL DEFAULT now()
);

CREATE UNIQUE INDEX users_email_lower_idx ON users (lower(email));

CREATE TRIGGER trg_users_updated_at
  BEFORE UPDATE ON users
  FOR EACH ROW EXECUTE FUNCTION set_updated_at();

-- ─── 3. expenses
─────────────────────────────────────────────────────────────

CREATE TABLE expenses (
  id            UUID         PRIMARY KEY,       -- client-generated
(ADR-003)
  user_id       UUID         NOT NULL REFERENCES users(id) ON DELETE
CASCADE,
  category_id   SMALLINT     NOT NULL REFERENCES categories(id) ON
DELETE RESTRICT,
  amount_minor  BIGINT       NOT NULL
                CHECK (amount_minor > 0 AND amount_minor <= 1000000000),
  currency_code CHAR(3)      NOT NULL DEFAULT 'INR'
                CHECK (currency_code ~ '^[A-Z]{3}$'),
  spent_on      DATE         NOT NULL,          -- ADR-002: calendar
date
  note          TEXT         CHECK (char_length(note) <= 200),
  created_at    TIMESTAMPTZ  NOT NULL DEFAULT now(),
  updated_at    TIMESTAMPTZ  NOT NULL DEFAULT now(),
  deleted_at    TIMESTAMPTZ                     -- NULL = live; non-null
= soft-deleted
);

CREATE TRIGGER trg_expenses_updated_at
  BEFORE UPDATE ON expenses
  FOR EACH ROW EXECUTE FUNCTION set_updated_at();

-- ─── 4. refresh_tokens
──────────────────────────────────────────────────────
```

```
CREATE TABLE refresh_tokens (
  id          UUID         PRIMARY KEY DEFAULT gen_random_uuid(),
  user_id     UUID         NOT NULL REFERENCES users(id) ON DELETE
CASCADE,
  token_hash  TEXT         NOT NULL,            -- hash of the actual
token
  family_id   UUID         NOT NULL,            -- shared by all
rotations from one sign-in
  expires_at  TIMESTAMPTZ  NOT NULL CHECK (expires_at > created_at),
  revoked_at  TIMESTAMPTZ,                      -- NULL = live; non-null
= revoked
  created_at  TIMESTAMPTZ  NOT NULL DEFAULT now(),
  UNIQUE (token_hash)
);

CREATE INDEX refresh_tokens_family_idx ON refresh_tokens (family_id);
CREATE INDEX refresh_tokens_user_idx   ON refresh_tokens (user_id);

-- ─── 5. usage_events
──────────────────────────────────────────────────────

CREATE TABLE usage_events (
  id          UUID         PRIMARY KEY DEFAULT gen_random_uuid(),
  user_id     UUID         REFERENCES users(id) ON DELETE SET NULL,
  event_type  TEXT         NOT NULL
              CHECK (event_type IN ('app_opened', 'expense_saved',
                                    'expense_deleted', 'summary_viewed',
                                    'export_requested')),
  platform    TEXT         CHECK (platform IN ('ios', 'android')),
  app_version TEXT,
  occurred_at TIMESTAMPTZ  NOT NULL DEFAULT now()
  -- Intentionally no amounts, categories, or notes (FR-10)
);

CREATE INDEX usage_events_user_occurred_idx
  ON usage_events (user_id, occurred_at);

-- ─── 6. deletion_requests
```

```
-- ------------------------------------------------------------

CREATE TABLE deletion_requests (
  user_id          UUID         PRIMARY KEY REFERENCES users(id) ON
DELETE CASCADE,
  requested_at     TIMESTAMPTZ  NOT NULL DEFAULT now(),
  grace_expires_at TIMESTAMPTZ  NOT NULL,  -- cancellation window end
  purge_after      TIMESTAMPTZ  NOT NULL,  -- job may purge after this
(NFR-8: 30 days)
  purged_at        TIMESTAMPTZ             -- set by job on completion
);

CREATE INDEX deletion_requests_purge_idx
  ON deletion_requests (purge_after) WHERE purged_at IS NULL;
```

Notes on the DDL worth reading. `categories.id` uses `GENERATED BY DEFAULT AS IDENTITY` — the SQL-standard auto-increment in PostgreSQL 10+, preferred over the legacy `SERIAL`. `BY DEFAULT` (not `ALWAYS`) allows explicit ID insertion during seeding. `users.id` uses `gen_random_uuid()` as a default; `expenses.id` has no DEFAULT because the client provides it (ADR-003) — this asymmetry is intentional. `users.pending_deletion_at` blocks sign-in immediately (compare to `now()`), while `deletion_requests` carries the full lifecycle; they work together. `grace_expires_at` and `purge_after` are separate columns because they serve different purposes: the first ends the cancellation window; the second controls when the job runs.

3.7 Index design

An index is a promise to pay a write cost on every insert, update, and delete in exchange for a read speedup on certain queries. Create an index when you can name the specific query it serves, and verify it helps with `EXPLAIN ANALYZE`.

Starting indexes for Spendwise:

Index	Query it serves	Notes
`expenses (user_id, spent_on) WHERE deleted_at IS NULL`	Monthly summary; list expenses for a month	Composite covers both the user filter and the date range. Partial index excludes deleted rows.

`expenses (user_id, category_id, spent_on) WHERE deleted_at IS NULL`	Category-filtered list; budget evaluation sum	Covering `category_id` lets the budget sum query filter by both user and category without a separate scan.
`users (lower(email))`	Sign-in lookup by email	Functional unique index; case-insensitive lookup and uniqueness in one.
`refresh_tokens (token_hash)`	Token validation at refresh	UNIQUE constraint creates this implicitly — no extra `CREATE INDEX` needed.
`refresh_tokens (family_id)`	Revoke all tokens in a family (theft detection)	Without this, revoking a family requires a sequential scan.
`refresh_tokens (user_id)`	Revoke all tokens for a user on sign-out or account deletion	Small table, but the habit transfers.
`usage_events (user_id, occurred_at)`	Usage reporting by user and time period	Append-heavy table.
`deletion_requests (purge_after) WHERE purged_at IS NULL`	The purge job finds accounts ready to delete	Without this the job does a full scan.

Indexes **NOT** created and why:

What a beginner might add	**Why we do not**
`expenses (deleted_at)` standalone	The partial index already handles live-only queries; a standalone index serves queries selecting deleted rows, which are rare.
`expenses (spent_on)` without `user_id`	No query scans expenses for a date range without a user filter. Put the higher-cardinality column first in a composite.

`expenses (note)`	No query searches by note text.
`expenses (amount_minor)`	No query filters by amount; aggregation is done after the user and date filter.
`categories (name)` standalone	The UNIQUE constraint on `name` already creates an index. Adding another is a duplicate.

The EXPLAIN habit. After the application runs with real queries, check `pg_stat_user_indexes` to find indexes with zero scans and `pg_stat_statements` to find slow queries. Remove unused indexes; they slow every write and waste I/O during autovacuum.

Creating indexes safely in production. Use `CREATE INDEX CONCURRENTLY` on a live table: it builds without an exclusive lock, at the cost of taking longer. Without `CONCURRENTLY`, the build holds an exclusive lock that blocks all reads and writes. For a ten-user pilot the window is tiny; build the habit now so it transfers to the table with ten million rows.

3.8 Schema change strategy

A schema in production is never finished. Two principles keep changes safe.

Principle 1: Expand before you contract. Adding a column is safe. Removing a column old code still reads is not. The safe sequence for renaming `note` to `description`:

1. Add `description TEXT` (expand).
2. Deploy code that writes to both columns.
3. Backfill: `UPDATE expenses SET description = note WHERE description IS NULL;`
4. Deploy code that reads from `description` only.
5. Remove `note` (contract).

This means old and new code can both run against the schema simultaneously, which is essential for rolling deployments and for mobile clients that update slowly.

Principle 2: Never rename a column directly. An existing column name is a contract with every deployed version of the app.

What is safe without special care: adding a nullable column; adding a column with a constant DEFAULT (PostgreSQL 11+: only a metadata change); adding a CHECK constraint; adding an index with `CONCURRENTLY`; adding a new table.

What requires care: adding a NOT NULL column without a DEFAULT (every existing row needs a value — provide a DEFAULT or backfill); adding a NOT NULL constraint to an existing nullable column (validates every row; slow on large tables); dropping a column (old code that names it fails — always expand-and-contract); changing a column type (may require a table rewrite; verify on a copy first); adding a FK to an existing column (PostgreSQL validates by scanning both tables).

Alembic. Each migration has `upgrade()` and `downgrade()`. Run migrations as a separate step before deploying new code. Never let the application run schema migrations automatically at startup in production; you lose control of timing and ordering.

3.9 Tracing requirements to schema

Every column should trace to a requirement, a decision, or a named pattern. This table does the tracing for the most important columns — the kind of review a tech lead does before approving a migration PR.

Column	Traces to
`expenses.id` (client UUID)	ADR-003: idempotent create
`expenses.amount_minor BIGINT`	ADR-002: money as minor units
`expenses.currency_code`	PRD assumption A2 (INR only); stored for future expansion
`expenses.spent_on DATE`	ADR-002: calendar date, not UTC timestamp
`expenses.deleted_at`	FR-3: deleted expenses excluded from totals
`expenses.note`	FR-2: expense can have an optional note
`users.pending_deletion_at`	TDD Flow 4: mark account pending deletion immediately
`deletion_requests.grace_expires_at`	Product decision: cancellation window
`deletion_requests.purge_after`	NFR-8: data deleted within 30 days of request

`refresh_tokens.family_id`	TDD Flow 3: token family revocation on theft detection
`refresh_tokens.revoked_at`	TDD Flow 3: tokens are revocable (sign-out, lost phone)
`usage_events.*` — no amounts, notes, categories	FR-10: no financial content in events
`categories.id SMALLINT`	TDD § 6: compact FK column; fixed list

Columns without a trace should be questioned. A common example is an `is_active` column added "in case we need it later." If no requirement uses it, do not add it; columns are cheaper to add than to remove.

Part 4 — Practical demonstration

4.1 The weak approach

Here is how a first-pass schema often looks:

```
CREATE TABLE users (
  id        SERIAL PRIMARY KEY,
  name      VARCHAR(255),
  email     VARCHAR(255),
  password  VARCHAR(255),
  created   TIMESTAMP,
  updated   TIMESTAMP
);

CREATE TABLE expenses (
  id         SERIAL PRIMARY KEY,
  user_id    INT,
  category   VARCHAR(100),
  amount     DECIMAL(10,2),
  date       DATE,
  note       VARCHAR(500),
  is_deleted BOOLEAN DEFAULT FALSE
);
```

Item by item, what is weak:

1. `id SERIAL`. Auto-increment integer breaks idempotent create from the client (ADR-003) and leaks row counts. There is no UUID for the client to carry in a retry.
2. `password VARCHAR(255)`. The column name suggests a plaintext password. It should be `password_hash`, nullable during the ADR-005 decision period.
3. `amount DECIMAL(10,2)`. Decimal is exact (better than `FLOAT`), but the minor-units pattern (ADR-002) is more explicit. `DECIMAL(10,2)` bakes in two decimal places, which fails for currencies like JPY (zero) or KWD (three).
4. `category VARCHAR(100)`. Stores the category name as a string on every row. No FK, no validation, no reference list. Two rows can disagree on "Food & Drink" vs "food".
5. `date DATE`. The column name `date` is a reserved word in most SQL dialects. Name it `spent_on`.
6. `note VARCHAR(500)`. The PRD says 200 characters. The schema allows 500. A business rule is not enforced.
7. `is_deleted BOOLEAN`. A boolean captures only "deleted or not," not when. `deleted_at TIMESTAMPTZ` gives the timestamp for auditing, for the purge job's 30-day calculation, and still evaluates as a null-check.
8. `created TIMESTAMP` and `updated TIMESTAMP`. Without TZ, these are ambiguous if the server's time zone setting ever changes.
9. No FK from `expenses.user_id` to `users.id`. No CASCADE. No integrity.
10. No CHECK constraints. No UNIQUE on email. No indexes.

Each item maps to a decision in Part 3. The decisions are not aesthetic preferences; each prevents a specific class of bug.

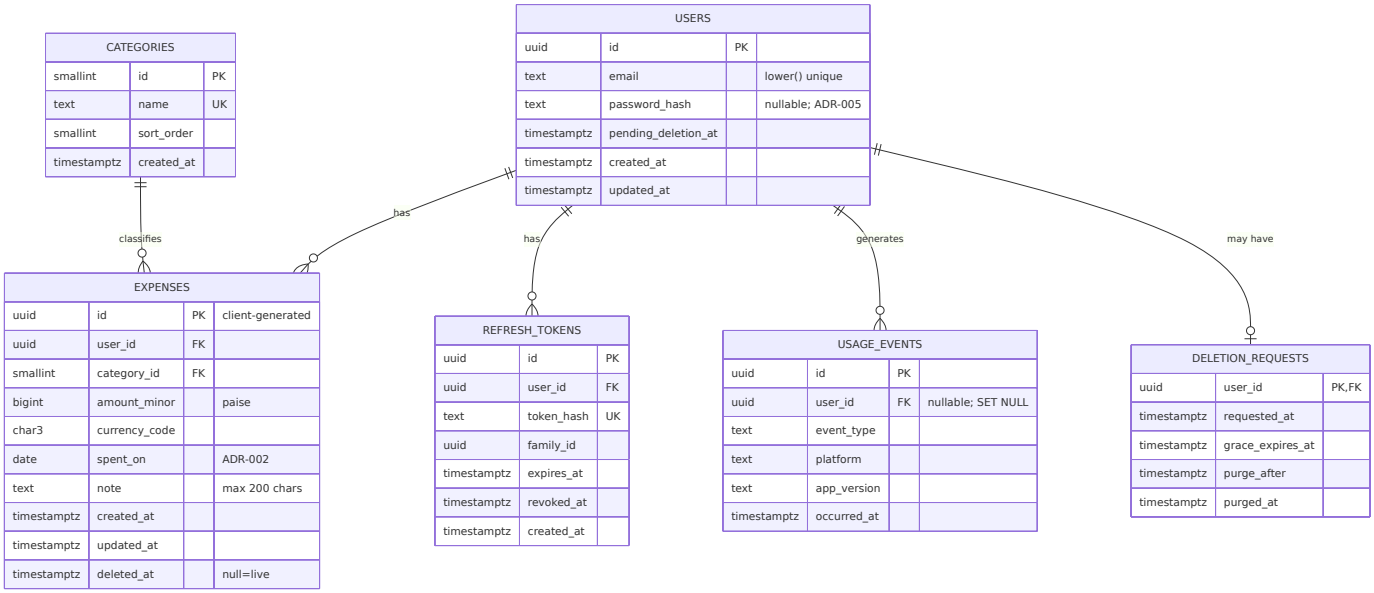
4.2 The stronger approach, step by step

1. Extract entities from requirements (section 3.2).
2. Identify relationships and choose ON DELETE behaviours (section 3.3).
3. For each column, choose the tightest correct type, decide nullability with a stated reason, and add CHECK constraints for business rules (section 3.4).
4. Decide how to represent categories: reference table, not ENUM, not inline CHECK (section 3.5).
5. Write the physical DDL (section 3.6).
6. Design indexes matched to real queries (section 3.7).

7. Verify each column traces to a requirement or named pattern (section 3.9).
8. Write the migration and verify it can be run safely on a live system.

4.3 The ERD diagram (Mermaid)

Keep this in `docs/erd.md` in the repository so it renders on GitHub or GitLab. Update it in the same PR as any migration that adds or removes a table or column. An outdated ERD is worse than none, because people trust it.



Part 5 — Documentation

What the ERD document contains:

1. The Mermaid (or PlantUML) diagram, checked into the repository.
2. The complete physical DDL as a reference snapshot (separate from the Alembic migration scripts, which are the authoritative source of how the schema was built).
3. A decisions table: for each non-obvious choice (`SMALLINT` for category IDs, partial indexes, ON DELETE behaviours), one line with the reason.
4. A column trace table: maps each column to the requirement or pattern that produced it.
5. A "what changes require a migration" note: helps future developers know which types of change are safe and which need a deploy procedure.

Where it lives. `docs/erd.md` with the diagram and decisions table; `docs/schema.sql` with the reference DDL. The Alembic migrations in `alembic/versions/` are the authoritative source of how the schema was built; the reference DDL is a convenience snapshot.

How it stays current. Every PR that adds a migration must also update `docs/erd.md` and `docs/schema.sql`. A CI check that compares the Alembic-generated schema against the reference file catches drift. For a solo project, a line in the PR template — "Did you update the ERD?" — is sufficient.

Part 6 — Trade-offs and failure scenarios

I'll label items **[Established]** where the practice is widely agreed and **[Context]** where your judgment decides.

6.1 SERIAL vs UUID vs IDENTITY [Context]

`SERIAL` (auto-increment integer) is simple and readable in queries. UUIDs are larger but globally unique, compatible with client-generation, and do not leak record counts. For a public-facing ID (expense ID returned to the client), UUID is correct because it does not reveal how many expenses exist in the system. For internal IDs that never leave the server (such as `refresh_token.id`), either works; I used UUID for consistency, which is a reasonable choice.

6.2 PostgreSQL ENUM types [Established]

PostgreSQL ENUM types look convenient but have a painful history. Before PostgreSQL 12, adding a new value required an exclusive lock. Even now, removing or renaming a value is not possible without workarounds. Use CHECK constraints or reference tables instead. This is one of those things that feels fine until the day you need to add "Cryptocurrency" as a category at 11 pm before a demo.

6.3 Forgetting WHERE deleted_at IS NULL [Established]

Soft delete is one of the most common sources of bugs. Every developer must remember to add the filter. Mitigations, in increasing strength:

1. One repository function for reading live expenses; all code calls it.
2. A partial index makes the database fast for the filtered query — a hint to developers that the filter matters.
3. PostgreSQL Row-Level Security can enforce the filter at the database level.
4. A view `live_expenses AS SELECT ... WHERE deleted_at IS NULL` can be queried instead of the base table.
5. A test that creates an expense, deletes it, and checks that it does not appear in the monthly summary.

All five mitigations together are not overkill for a finance app.

6.4 ON DELETE and data compliance [Context]

`ON DELETE CASCADE` is correct for `expenses → users` if account deletion must erase financial records (NFR-8). But even after a cascade delete, the data lives in backups until they expire (at most 35 days per NFR-8). This must be communicated to the client as part of the compliance story. If a different jurisdiction requires immediate deletion from backups, the architecture changes fundamentally — perhaps requiring per-user encryption keys that can be destroyed ("crypto shredding").

6.5 The updated_at trigger vs application responsibility [Context]

The trigger approach guarantees `updated_at` is set even when a developer forgets. The trade-off: it is less visible in code reviews, adds a small overhead per write, and requires knowing that triggers exist. The application-level approach (the service always sets `updated_at`) is more visible but more brittle. For a solo project, the trigger wins; for a team, document the trigger prominently.

6.6 Functional index for case-insensitive email [Context]

`CREATE UNIQUE INDEX users_email_lower_idx ON users (lower(email))` is correct. But it means queries must also use `WHERE lower(email) = lower(:input)` to hit the index. If a developer writes `WHERE email = :input` (case-sensitive), the query still works but uses a sequential scan. Mitigation: enforce the `lower()` in the repository function; add a CI test that queries by email with mixed case and checks the execution plan.

6.7 Token hash storage [Established]

Storing refresh tokens as plaintext in the database means a database breach exposes all active sessions. Storing the hash (SHA-256 is sufficient for a high-entropy random token) means a breach exposes unusable values. The token itself is sent to the client, and the client sends it back; the server hashes and compares. Never store any token, password, or secret as plaintext.

6.8 Token family for theft detection [Context]

The `family_id` column groups all rotations from a single sign-in event. When the API receives a refresh token it has already rotated (a "stale token"), it can conclude the token was either replayed (a bug) or stolen. Revoking the entire family logs out all legitimate sessions

from that sign-in, which is aggressive but safe. The alternative — revoke only the stale token — fails if the attacker rotated before the legitimate client did. The appropriate level of aggression depends on the threat model.

6.9 Migrations that lock production [Established]

A migration that rewrites a large table or builds an index without `CONCURRENTLY` can block writes. With ten users nobody notices. With a thousand, a lunch-time migration becomes an outage. Build the habit now, on a small table, when the stakes are low.

Part 7 — Hands-on exercise

This exercise extends the budget alerts tables from the Lecture 1 exercise into a full physical ERD. You should produce a result that can be reviewed alongside the base Spendwise schema.

Context (from Lecture 1 exercise reference solution)

Two tables were sketched:

```
CREATE TABLE budgets (
  user_id, category_id, limit_minor, currency_code, created_at,
updated_at
  PRIMARY KEY (user_id, category_id)
);

CREATE TABLE budget_alerts (
  id, user_id, category_id, month_start DATE, threshold SMALLINT,
  triggered_by_expense_id UUID, fired_at,
  UNIQUE (user_id, category_id, month_start, threshold)
);
```

The sketched DDL is correct in spirit but incomplete. Your task is to produce the production-ready DDL.

Tasks (about 60 minutes)

Task 1: Column audit (15 min). For both tables, list every column. For each column state: (a) is the type correct for PostgreSQL? (b) is the nullability correct? (c) is there a DEFAULT

missing? (d) is there a CHECK constraint that should be added? Add `updated_at` to `budgets` and explain why `budget_alerts` does not need it.

Task 2: FK audit (10 min). Every foreign key in both tables should specify ON DELETE behaviour. State what you choose for each FK and why. There is one FK where the correct choice depends on a product question you need to name.

Task 3: Index audit (10 min). The Lecture 1 reference solution created two indexes. Are they correct? Are any missing? Are any that could be replaced with the UNIQUE constraint's implicit index? Verify each against a real query.

Task 4: Migration strategy (10 min). These tables are new, so there is no existing table to worry about. Describe what you would add in a future migration if the product later required: (a) a third threshold at 50%; (b) a budget that rolls over month to month; (c) a budget that covers multiple categories.

Task 5: Production-ready DDL (15 min). Write the complete DDL for both tables, incorporating corrections from tasks 1–3, the `set_updated_at` trigger where appropriate, and comments where a decision needs documentation.

Task 6 (challenge). Write a single query that returns, for the current user, all categories with a budget, the limit, the amount spent this month, and the highest threshold that has already fired. The query should be usable as the source for both the monthly summary budget display and the save-expense response.

Part 8 — Review of a typical attempt

Here is a common first-pass answer to the exercise, along with the review.

Typical attempt:

```
CREATE TABLE budgets (
  id UUID PRIMARY KEY,
  user_id UUID NOT NULL REFERENCES users(id),
  category_id INT NOT NULL REFERENCES categories(id),
  limit_amount DECIMAL(12,2) NOT NULL,
  currency VARCHAR(3),
  created_at TIMESTAMP,
  updated_at TIMESTAMP
);

CREATE TABLE budget_alerts (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  user_id UUID NOT NULL REFERENCES users(id),
  category_id INT,
  month DATE,
  threshold INT,
  expense_id UUID,
  created_at TIMESTAMP
);
```

What was done well: UUIDs as primary keys, a separate `id` for `budget_alerts`, references to users and categories.

Gap 1: `budgets` has a surrogate `id` but should use a composite PK. A budget is uniquely identified by `(user_id, category_id)` — the business rule "one budget per category per user" is the natural key. Adding a UUID surrogate means you need a UNIQUE constraint on `(user_id, category_id)` anyway, maintaining two things that serve the same purpose. Use the composite PK and remove `id`. The Lecture 1 reference solution already showed this.

Gap 2: `category_id INT` but the base schema uses `SMALLINT`. Mismatched types on a FK column cause an implicit cast on every join, prevent the planner from choosing the most efficient plan, and will cause a migration error in PostgreSQL if the FK column type does not match the referenced column type. Always match the type of the referenced column exactly.

Gap 3: `limit_amount DECIMAL(12,2)`. ADR-002 chose `BIGINT` minor units for all money values. If `expenses.amount_minor` is in paise, then `budgets.limit_minor` must also be in paise. Mixing `DECIMAL` on the budget with `BIGINT` on the expense means conversion

logic in the comparison code, and one missed conversion is a bug that shows wrong thresholds.

Gap 4: `currency VARCHAR(3)` nullable with no DEFAULT. Should be `currency_code CHAR(3) NOT NULL DEFAULT 'INR'`, consistent with `expenses.currency_code`.

Gap 5: `TIMESTAMP` without TZ. All timestamps in the schema use `TIMESTAMPTZ`. This is not an aesthetic preference; see section 3.4.

Gap 6: `budget_alerts.month DATE`. The column name `month` is ambiguous (is it the first of the month? any date?). The correct name is `month_start DATE NOT NULL`, and a CHECK constraint enforces it: `CHECK (EXTRACT(DAY FROM month_start) = 1)`. The Lecture 1 reference solution showed this.

Gap 7: No UNIQUE constraint on `budget_alerts`. The entire guarantee of "at most one alert per threshold per category per month" rests on `UNIQUE (user_id, category_id, month_start, threshold)`. Without it, `ON CONFLICT DO NOTHING` silently degrades to a regular insert that creates duplicates.

Gap 8: `threshold INT` with no CHECK. The only valid values are 80 and 100 (for v1). `CHECK (threshold IN (80, 100))` enforces this at the database level.

Gap 9: No ON DELETE specification on any FK. The behaviour defaults to `NO ACTION`. For `budgets → users`: `CASCADE`. For `budget_alerts → users`: `CASCADE`. For `budget_alerts → expenses (expense_id)`: `SET NULL` — the alert record survives even if the triggering expense is later hard-deleted; the link is informational, not structural.

Gap 10: No indexes. The critical query — `SELECT SUM(amount_minor) FROM expenses WHERE user_id = :u AND category_id = :c AND spent_on >= :start AND ...` — already has an index (`expenses_user_cat_spent_idx` from Lecture 1). The budget fetch uses the composite PK, which is implicitly indexed.

Overall verdict. The types are partially wrong, nullability is inconsistent, business-rule constraints are missing, and the single most important guarantee (the UNIQUE constraint) is absent. Each gap maps directly to a section in Part 3.

Part 8B — Reference solution

Task 1: Column audit answers

`budgets`:

- `limit_minor BIGINT NOT NULL CHECK (limit_minor > 0 AND limit_minor <= 1000000000)` — matches `expenses.amount_minor`;
- `currency_code CHAR(3) NOT NULL DEFAULT 'INR' CHECK (currency_code ~ '^[A-Z]{3}$')`;
- `created_at TIMESTAMPTZ NOT NULL DEFAULT now()`;
- `updated_at TIMESTAMPTZ NOT NULL DEFAULT now()` — needed because users can change their budget limit; set by trigger.

`budget_alerts`:

- `month_start DATE NOT NULL CHECK (EXTRACT(DAY FROM month_start) = 1)` — enforces first-of-month invariant;
- `threshold SMALLINT NOT NULL CHECK (threshold IN (80, 100))` — `SMALLINT`, not `INT`;
- `triggered_by_expense_id UUID REFERENCES expenses(id) ON DELETE SET NULL` — nullable; `SET NULL` so alerts survive if the expense is purged;
- `fired_at TIMESTAMPTZ NOT NULL DEFAULT now()`.

No `updated_at`: alerts are immutable once created. Adding it would imply they can be edited, which is misleading.

Task 2: FK audit answers

FK	ON DELETE	Reason
`budgets.user_id → users.id`	`CASCADE`	Budget deleted with account
`budgets.category_id → categories.id`	`RESTRICT`	Cannot delete a category with a budget
`budget_alerts.user_id → users.id`	`CASCADE`	Alerts deleted with account
`budget_alerts.category_id → categories.id`	`RESTRICT`	Mirrors expenses

`budget_alerts.triggered_by_expense_id → expenses.id`	`SET NULL`	Alert survives if expense is later purged; link is informational

The product question: if a budget is deleted, should existing alerts be deleted too, or kept for the user's records? The sketch has no FK from `budget_alerts` to `budgets`. If you add one, the ON DELETE choice — `RESTRICT` or `CASCADE` — is a product decision to bring to Meera.

Task 3: Index audit answers

From the Lecture 1 exercise:

- `expenses (user_id, category_id, spent_on) WHERE deleted_at IS NULL` — correct; used by the budget sum query.
- `budget_alerts (triggered_by_expense_id)` — correct; used by the retry replay path.

The UNIQUE constraint on `(user_id, category_id, month_start, threshold)` creates an implicit index covering the on-conflict insert. No additional index needed for `budgets` because the composite PK covers the `FOR UPDATE` row lock.

Task 4: Migration strategy for future changes

(a) Third threshold at 50%.

```
ALTER TABLE budget_alerts DROP CONSTRAINT budget_alerts_threshold_check;
ALTER TABLE budget_alerts ADD CONSTRAINT budget_alerts_threshold_check
  CHECK (threshold IN (50, 80, 100));
```

A non-destructive drop-and-re-add. Holds a short lock but does not rewrite the table.

(b) Rollover budgets. Add `rollover BOOLEAN NOT NULL DEFAULT FALSE` to `budgets`, and a nullable `rollover_amount_minor BIGINT` for the carried amount. No structural change to `budget_alerts`.

(c) Multi-category budgets. The natural key `(user_id, category_id)` on `budgets` enforces one budget per category. Multi-category budgets require a different data model: a `budget_groups` table (one row per named group) and a `budget_group_categories` junction table. This is an M:N change that cannot be done by adding columns to the existing table.

Task 5: Production-ready DDL

```
-- ─── budgets
──────────────────────────────────────────────────────────────

CREATE TABLE budgets (
  user_id       UUID     NOT NULL REFERENCES users(id) ON DELETE
CASCADE,
  category_id   SMALLINT NOT NULL REFERENCES categories(id) ON DELETE
RESTRICT,
  limit_minor   BIGINT   NOT NULL
                CHECK (limit_minor > 0 AND limit_minor <= 1000000000),
  currency_code CHAR(3)  NOT NULL DEFAULT 'INR'
                CHECK (currency_code ~ '^[A-Z]{3}$'),
  created_at    TIMESTAMPTZ NOT NULL DEFAULT now(),
  updated_at    TIMESTAMPTZ NOT NULL DEFAULT now(),
  PRIMARY KEY (user_id, category_id)    -- one limit per category per
user
);

CREATE TRIGGER trg_budgets_updated_at
  BEFORE UPDATE ON budgets
  FOR EACH ROW EXECUTE FUNCTION set_updated_at();

-- ─── budget_alerts
─────────────────────────────────────────────────────────

CREATE TABLE budget_alerts (
  id                    UUID     PRIMARY KEY DEFAULT
gen_random_uuid(),
  user_id               UUID     NOT NULL REFERENCES users(id) ON
DELETE CASCADE,
  category_id           SMALLINT NOT NULL REFERENCES categories(id) ON
DELETE RESTRICT,
  month_start           DATE     NOT NULL
                        CHECK (EXTRACT(DAY FROM month_start) = 1),
  threshold             SMALLINT NOT NULL CHECK (threshold IN (80,
100)),
  triggered_by_expense_id UUID     REFERENCES expenses(id) ON DELETE SET
NULL,
  fired_at              TIMESTAMPTZ NOT NULL DEFAULT now(),
```

```
  -- Guarantee: at most one alert per threshold per category per month
per user
  UNIQUE (user_id, category_id, month_start, threshold)
);

CREATE INDEX budget_alerts_trigger_idx
  ON budget_alerts (triggered_by_expense_id);
```

Task 6 (challenge): Budget status query

```
SELECT
  c.id                                              AS category_id,
  c.name                                            AS category_name,
  b.limit_minor,
  COALESCE(e.spent_minor, 0)                        AS spent_minor,
  b.limit_minor - COALESCE(e.spent_minor, 0)        AS remaining_minor,
  ROUND(
    COALESCE(e.spent_minor, 0)::NUMERIC * 100
    / b.limit_minor
  )                                                 AS pct_used,
  MAX(a.threshold)                                  AS
highest_alert_fired
FROM budgets b
JOIN categories c ON c.id = b.category_id
LEFT JOIN (
  SELECT category_id, SUM(amount_minor) AS spent_minor
  FROM expenses
  WHERE user_id    = :uid
    AND spent_on  >= :month_start
    AND spent_on   < :next_month_start
    AND deleted_at IS NULL
  GROUP BY category_id
) e ON e.category_id = b.category_id
LEFT JOIN budget_alerts a
  ON  a.user_id     = b.user_id
  AND a.category_id = b.category_id
  AND a.month_start = :month_start
WHERE b.user_id = :uid
GROUP BY c.id, c.name, b.limit_minor, e.spent_minor;
```

Notes: the inner aggregate on `expenses` runs once (not per budget row), joined to the budgets. `MAX(threshold)` returns the highest threshold fired or `NULL` if none. The division by `b.limit_minor` is safe because `CHECK (limit_minor > 0)` is enforced. The `::NUMERIC` cast before multiplication prevents integer truncation.

Part 9 — Recall test, with answers

Cover the answers and try first.

Q1. What are the three levels of ERD? Which level does a lead produce before writing migration scripts?

Answer: Conceptual (entities and relationships only), logical (attributes and generic types), physical (PostgreSQL-specific types, nullability, constraints, indexes). A lead produces the physical ERD.

Q2. Why is `amount DECIMAL(10,2)` a weaker choice than `amount_minor BIGINT` for storing money?

Answer: `DECIMAL` is exact (unlike `FLOAT`), but it bakes in two decimal places, which fails for currencies with different conventions (JPY: 0, KWD: 3). `BIGINT` with an explicit `currency_code` and a documented divisor is both exact and explicit. Both are technically correct; `BIGINT` minor units is the more professional convention. `FLOAT` for money is simply wrong.

Q3. Why use `TIMESTAMPTZ` rather than `TIMESTAMP` for audit columns?

Answer: `TIMESTAMP` stores a bare clock reading with no time zone. If the server's time zone setting changes, all stored values are misinterpreted. `TIMESTAMPTZ` stores UTC internally and displays in the session time zone, making the value unambiguous regardless of where the server is deployed.

Q4. What is the difference between `ON DELETE CASCADE` and `ON DELETE SET NULL`, and when would you use each?

Answer: `CASCADE` deletes the child row when the parent is deleted. `SET NULL` sets the FK column to `NULL` in the child row. Use `CASCADE` when the child cannot meaningfully exist without the parent (expenses without a user). Use `SET NULL` when the child should survive but the reference should become empty (usage events retained for aggregate analytics after an account is deleted).

Q5. Why does the `categories` table use `GENERATED BY DEFAULT AS IDENTITY` rather than `SERIAL`?

Answer: `GENERATED BY DEFAULT AS IDENTITY` is the SQL-standard equivalent of a sequence-backed auto-increment, introduced in PostgreSQL 10. `SERIAL` is legacy syntax that creates an implicit sequence with a less explicit ownership model. `BY DEFAULT` (not

`ALWAYS`) allows explicit ID insertion during seeding, which is useful when migrating an existing dataset.

Q6. What does a partial index `WHERE deleted_at IS NULL` do, and what is its advantage over a full index?

Answer: A partial index only includes rows where `deleted_at IS NULL` (live expenses). Because deleted expenses are excluded from all queries that use the index, the index is smaller (fewer rows, less disk and memory), faster to scan, and faster to maintain on writes. Queries that do not filter by `deleted_at IS NULL` will not use the partial index, which is acceptable because those queries are rare.

Q7. What is the `family_id` column on `refresh_tokens` for?

Answer: It groups all rotations from a single sign-in event into a "family." If a previously rotated (stale) token is presented to the API, the API treats this as evidence of token theft and revokes all tokens in the family, logging out all sessions from that sign-in. Without family tracking, revocation would affect only the stale token, potentially leaving a stolen rotated token still valid.

Q8. What is the expand-and-contract pattern, and when do you need it?

Answer: A safe way to rename a column on a live system. Expand: add the new column, deploy code that writes to both. Contract: backfill, deploy code that reads from the new column only, remove the old column. This ensures old and new code can run simultaneously against the schema, which is required for rolling deployments and for mobile clients that update slowly.

Q9. Why does `budget_alerts.month_start` have a CHECK constraint that the day equals 1?

Answer: `month_start` is supposed to always hold the first day of a month (e.g. `2026-10-01`). Without the CHECK, a bug could insert `2026-10-15`, and the UNIQUE constraint would not catch it (it would allow a second alert for October at threshold 80, as long as the `month_start` value differs). The CHECK enforces the invariant at the database level so no code path can insert a bad value.

Q10 (challenge). You have a table with 50 million rows that needs an index on `(user_id, category_id)`. How do you add it without taking the table offline, and what are the trade-offs?

Answer: `CREATE INDEX CONCURRENTLY idx_name ON table (user_id, category_id);` `CONCURRENTLY` builds the index without an exclusive lock; it makes two passes over the table while the table is in normal use. Trade-offs: the build takes longer (roughly twice), uses more I/O during the build, and if the operation is cancelled or fails, it leaves an invalid index that must be manually dropped and rebuilt. Check `pg_stat_progress_create_index` to monitor progress. Never use `CREATE INDEX` without `CONCURRENTLY` on a table with live write traffic.

Part 10 — Completion checklist

Tick honestly. Revisit the section noted for any you cannot tick.

- [ ] I can name the three ERD levels and explain what the physical ERD contains that the others do not (3.1).
- [ ] I can read a set of requirements and identify the entities, relationships, and rules they imply, and trace each entity to a requirement (3.2).
- [ ] I can choose the correct ON DELETE behaviour for each foreign key and explain the reasoning (3.3).
- [ ] I can select the right PostgreSQL type for money, dates, text, UUIDs, and small integers, and explain why the common alternatives are wrong (3.4).
- [ ] I can apply the nullability rule (NOT NULL by default; NULL with a stated reason) to every column in a schema (3.4).
- [ ] I can write CHECK constraints for business rules that apply to a single row and say what does NOT belong in a CHECK constraint (3.4).
- [ ] I can explain the three options for representing a fixed list (ENUM, CHECK, reference table) and choose the correct one (3.5).
- [ ] I can write the full physical DDL for the Spendwise schema from first principles, including the trigger function, all constraints, and all starting indexes (3.6).
- [ ] I can design a set of starting indexes matched to real queries, using partial indexes where appropriate, and explain what would not be indexed and why (3.7).
- [ ] I can describe the expand-and-contract pattern and identify which schema changes are safe on a live system and which require the pattern (3.8).
- [ ] I can trace every column in a schema to a requirement, a decision, or a named pattern, and question columns that cannot be traced (3.9).
- [ ] I can review a peer's schema and identify missing constraints, wrong types, nullable columns without a reason, and missing or extraneous indexes (Part 4, Part 8).

Closing: what you should understand before moving on

Before Lecture 3 (the API contract), make sure you can say these five things in your own words:

1. **The physical ERD is a contract.** Every column, type, and constraint is a commitment to every developer and every query. "We'll figure out the columns later" is not a plan.
2. **Types are not cosmetic.** `BIGINT` vs `DECIMAL`, `DATE` vs `TIMESTAMPTZ`, `SMALLINT` vs `UUID` — each choice affects correctness, storage, index efficiency, and the clarity of the schema for future readers.
3. **Constraints are the last line of defence.** Application code will have bugs. The CHECK constraint, the UNIQUE constraint, and the NOT NULL constraint enforce the rule even when the code is wrong. Build them in from the start, on every table.
4. **Indexes serve real queries.** Create an index when you can name the query it speeds up. Verify it with `EXPLAIN ANALYZE`. Remove it if `pg_stat_user_indexes` shows zero scans after real usage.
5. **Every column should trace to something.** A column with no requirement behind it is future maintenance debt. The discipline of tracing keeps schemas lean, readable, and defensible in code review.

This is the end of Module 3, Lecture 2. I'm stopping here, as instructed.